

- Description

This module manages an automatic speed limit adjustment system for smart highways, ensuring that:

1. The system continuously monitors the current traffic density (in vehicles per kilometer).
2. If the traffic density exceeds a predefined threshold, the speed limit is reduced to prevent congestion and improve safety.
3. If the traffic density is below the threshold, the speed limit remains unchanged.

This system helps optimize traffic flow and reduce the risk of accidents by dynamically adjusting speed limits based on real-time traffic conditions.

- Specification

module SmartHighwaySpeedLimitControl

var currentTrafficDensity, maxTrafficDensity: real

var normalSpeedLimit, reducedSpeedLimit, currentSpeedLimit: integer

process updateTrafficDensity(newTrafficDensity: real) returns (currentSpeedLimit: integer)

functional scenario form:

currentTrafficDensity > maxTrafficDensity && currentSpeedLimit = reducedSpeedLimit, output "ALERT: High traffic density (" + currentTrafficDensity + " vehicles/km). Speed limit reduced to " + currentSpeedLimit + " km/h.", Speed limit reduced.

||

currentTrafficDensity <= maxTrafficDensity && currentSpeedLimit = normalSpeedLimit, output "Traffic density normal (" + currentTrafficDensity + " vehicles/km). Speed limit remains " + currentSpeedLimit + " km/h.", Speed limit remains normal.

end process

end module

- Java Code

```
package code;
public class c97_SmartHighway {
    private double maxTrafficDensity;
    private double currentTrafficDensity;
    private int normalSpeedLimit;
    private int reducedSpeedLimit;
    private int currentSpeedLimit;
    public c97_SmartHighway(double maxTrafficDensity, int normalSpeedLimit, int reducedSpeedLimit) {
        this.maxTrafficDensity = maxTrafficDensity;
        this.normalSpeedLimit = normalSpeedLimit;
        this.reducedSpeedLimit = reducedSpeedLimit;
    }
}
```

```

        this.currentTrafficDensity = 0.0;
        this.currentSpeedLimit = normalSpeedLimit;
    }
    public void updateTrafficDensity(double newTrafficDensity) {
        currentTrafficDensity = newTrafficDensity;
        adjustSpeedLimit();
        System.out.printf("Current Traffic Density: %.2f vehicles/km, S
peed Limit: %d km/h\n",
            currentTrafficDensity, currentSpeedLimit);
        if (currentTrafficDensity > maxTrafficDensity) {
            System.out.println("ALERT: High traffic detected. Speed lim
it reduced.");
        } else {
            System.out.println("Traffic conditions normal. Speed limit
unchanged.");
        }
    }
    private void adjustSpeedLimit() {
        if (currentTrafficDensity > maxTrafficDensity) {
            currentSpeedLimit = reducedSpeedLimit;
        } else {
            currentSpeedLimit = normalSpeedLimit;
        }
    }
    public int getCurrentSpeedLimit() {
        return currentSpeedLimit;
    }
    public static void main(String[] args) {
        c97_SmarthHighway highway = new c97_SmarthHighway(50.0, 120, 80);
        // testcase-VT:
        //     highway.updateTrafficDensity(30.0);
        //     highway.updateTrafficDensity(60.0);
        //     highway.updateTrafficDensity(45.0);

        // testcase-FT:
        //     highway.updateTrafficDensity(0.0);
        //     highway.updateTrafficDensity(9.063142030601998E290);
        //     highway.updateTrafficDensity(-1.7976931348623157E308);

        // testcase-Z3:
        //     highway.updateTrafficDensity(30.0);
        //     highway.updateTrafficDensity(60.0);
        //     highway.updateTrafficDensity(45.0);
    }
}

```

```

// testcase-UVT:
highway.updateTrafficDensity(30.0);
highway.updateTrafficDensity(60.0);
highway.updateTrafficDensity(45.0);
}
}

```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c97_SmartHighway.java	100.0 %	4	0	4
▼ c97_SmartHighway	100.0 %	4	0	4
● updateTrafficDensity(double)	100.0 %	2	0	2
■ adjustSpeedLimit()	100.0 %	2	0	2
● c97_SmartHighway(double, int, i		0	0	0
● main(String[])		0	0	0

3. Vibration testing instruction coverage: 96.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	88	21,672	21,760
▼ src	0.4 %	88	21,672	21,760
▼ code	0.4 %	88	21,672	21,760
▼ c97_SmartHighway.java	96.7 %	88	3	91
▼ c97_SmartHighway	96.7 %	88	3	91
● updateTrafficDensity(double)	100.0 %	37	0	37
■ adjustSpeedLimit()	100.0 %	16	0	16
● c97_SmartHighway(double, int, i	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17

4. Fuzz testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c97_SmartHighway.java	100.0 %	4	0	4
▼ c97_SmartHighway	100.0 %	4	0	4
● updateTrafficDensity(double)	100.0 %	2	0	2
■ adjustSpeedLimit()	100.0 %	2	0	2
● c97_SmartHighway(double, int, i		0	0	0
● main(String[])		0	0	0

5. Fuzz testing instruction coverage: 96.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	88	21,672	21,760
▼ src	0.4 %	88	21,672	21,760
▼ code	0.4 %	88	21,672	21,760
▼ c97_SmartHighway.java	96.7 %	88	3	91
▼ c97_SmartHighway	96.7 %	88	3	91
● updateTrafficDensity(double)	100.0 %	37	0	37
■ adjustSpeedLimit()	100.0 %	16	0	16
● c97_SmartHighway(double, int, i	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c97_SmartHighway.java	100.0 %	4	0	4
▼ c97_SmartHighway	100.0 %	4	0	4
● updateTrafficDensity(double)	100.0 %	2	0	2
■ adjustSpeedLimit()	100.0 %	2	0	2
● c97_SmartHighway(double, int, i		0	0	0
● main(String[])		0	0	0

7. Z3 instruction coverage: 96.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	88	21,672	21,760
▼ src	0.4 %	88	21,672	21,760
▼ code	0.4 %	88	21,672	21,760
▼ c97_SmartHighway.java	96.7 %	88	3	91
▼ c97_SmartHighway	96.7 %	88	3	91
● updateTrafficDensity(double)	100.0 %	37	0	37
■ adjustSpeedLimit()	100.0 %	16	0	16
● c97_SmartHighway(double, int, i	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c97_SmartHighway.java	100.0 %	4	0	4
▼ c97_SmartHighway	100.0 %	4	0	4
● updateTrafficDensity(double)	100.0 %	2	0	2
■ adjustSpeedLimit()	100.0 %	2	0	2
● c97_SmartHighway(double, int, i		0	0	0
● main(String[])		0	0	0

9. Unimproved V-method instruction coverage: 96.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	88	21,672	21,760
▼ src	0.4 %	88	21,672	21,760
▼ code	0.4 %	88	21,672	21,760
▼ c97_SmartHighway.java	96.7 %	88	3	91
▼ c97_SmartHighway	96.7 %	88	3	91
● updateTrafficDensity(double)	100.0 %	37	0	37
■ adjustSpeedLimit()	100.0 %	16	0	16
● c97_SmartHighway(double, int, i	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17